

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Debugging & Optimization Task

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 2 – Debugging & Optimisation Task
Weightage:	35% of CIA	Total Marks:	100
CO2 Statement:	Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.		
Student Name:	V YESHWNATH	Reg. No.:	192511300
Section / Batch:		Date:	22/07/2026

Code of Conduct

I V YESHWANTH (192511300) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the Student: V YESHWANTH

Instructions

- This test contains 20 Debugging & Optimisation Task questions. Total: 100 Marks (20*5).
- When a student answer using Scenario-Based, in evaluation the answer should be with following five requirements: Identify the problem type and constraints, Select appropriate data structure, Design the Solution, Optimization and efficiency, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Search Data Structure, Stack	Binary Search, Stack Operations	1 - 5	25
Linked List Data Structure	List Operations	6 – 10	25
Queue Data Structures	Queue Operations	11 - 15	25
Queue Data Structures, Stack Notations	Queue Operations, Expression Evaluation	16 - 20	25
GRAND TOTAL:		100 Marks	

0/2 answered

CO3_AT3_MCQ OPEN ✓ Completed

Multiple Choice Questions

25/25 answered

Review →

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2) OPEN ✓ Completed

Debugging & Optimization Task

10/10 answered

Review →

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 1) OPEN ✓ Completed

Debugging & Optimization Task

10/10 answered

Review →

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 1)

← Back

QUESTIONS

Q1 Binary Search

5 marks

Q2 Stack

5 marks

Q3 Stack

5 marks

Q4 Stack

5 marks

Q5 Stack

5 marks

Q6 Linked List

5 marks

Q7 Stack

5 marks

Q8 Linked List Data Structure

5 marks

Q9 Linked List

5 marks

Q10 Queue

5 marks

10/10 answered

Binary Search

5 marks

Identify the error in the following snippet.

```
int binarySearch(int arr[], int n, int key) {  
    int low = 0, high = n - 1;  
    while(low < high) {  
        int mid = (low + high) / 2;  
        if(arr[mid] == key)  
            return mid;  
        else if(arr[mid] < key)  
            low = mid;  
        else  
            high = mid;  
    }  
    return -1;  
}
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
#include <stdio.h>  
#include <stdlib.h>  
  
int binarySearch(int arr[], int n, int key)  
{  
    int low = 0, high = n - 1;  
    while (low <= high)  
    {  
        int mid = (low + high) / 2;  
        if (arr[mid] == key)  
            return mid;  
        else if (arr[mid] < key)  
            low = mid + 1;  
        else  
            high = mid - 1;  
    }  
    return -1;  
}  
  
int main()  
{  
    int n, key;  
    scanf("%d", &n);  
    int arr[n];  
    for (int i = 0; i < n; i++)  
        scanf("%d", &arr[i]);  
    scanf("%d", &key);  
    int result = binarySearch(arr, n, key);  
    if (result != -1)  
        printf("%d", result);  
}
```

Test Input

5
10 20 30 40 50
30

Output

Pending manual review

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

← Back

QUESTIONS

- Q1
Stack Notations - Prefix Expression
5 marks
- Q2
Circular Queue
5 marks
- Q3
Binary Search
5 marks
- Q4
Queue
5 marks
- Q5
Queue
5 marks
- Q6
Doubly Linked List
5 marks
- Q7
Stack
5 marks
- Q8
Stack
5 marks
- Q9
Circular Queue
5 marks
- Q10
Stack
5 marks

10/10 answered

Stack Notations - Prefix Expression

5 marks

Find the bug in the evaluation of a Prefix expression.

```
while (expression[i] != '\0') {  
    if (isdigit(expression[i])) push(expression[i] - '0');  
    else {  
        int op1 = pop();  
        int op2 = pop();  
        // perform operation: result = op1 + op2;  
        push(result);  
    }  
}
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

Test Input

stdin input (optional)

Output

```
1 int i = 0;  
2 while (expression[i] != '\0') {  
3     // If the character is a digit, convert to integer and push to stack  
4     if (isdigit(expression[i])) {  
5         push(expression[i] - '0');  
6     }  
7     // Skip spaces if the expression contains them  
8     else if (expression[i] == ' ') expression[i] = '\0';  
9     i++;  
10    continue;  
11 }  
12 // If the character is an operator  
13 else {  
14     int op1 = pop(); // First popped is the second operand  
15     int op2 = pop(); // Second popped is the first operand  
16     int result = 0;  
17     switch (expression[i]) {  
18         case '+': result = op1 + op2; break;  
19         case '-': result = op1 - op2; break;  
20         case '*': result = op1 * op2; break;  
21         case '/': result = op1 / op2; break;  
22     }  
23     push(result);  
24 }  
25 i++; // Fix: Increment index to move to the next character  
26 }  
27 }
```

Run

Save

Submit Fix

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Debugging Accuracy	20	All errors corrected; code runs flawlessly	Most errors corrected; minor issues remain	Partial correction; code runs with issues	Code fails or major errors persist
Error Identification	30	Accurately identifies all logical, syntax, and edge-case errors	Identifies most major errors	Identifies some errors but misses key issues	Struggles to identify errors
Code Quality & Readability	20	Clean, modular, well documented, follows best practices	Mostly clean and readable	Some structure but inconsistent	Poorly written, hard to understand
Explanation & Justification	20	Clear, logical, and well justified reasoning with complexity analysis	Good explanation with some justification	Limited explanation; lacks depth	No or incorrect explanation
Testing & Validation	10	Comprehensive test cases including edge cases	Adequate test cases	Minimal testing	No testing provided
Faculty In-charge		Course Coordinator		Program Director	
Signature: _____		Signature: _____		Signature: _____	
Date: _____		Date: _____		Date: _____	